DIAGNOSTIC & CORRECTION DE FUITES MÉMOIRE EN C

Analyse bas niveau, traçage dynamique et sécurisation du code (Valgrind & GDB)



LE SOMMAIRE

- CONTEXTE & OBJECTIFS
- LA BOÎTE À OUTILS (ENVIRONNEMENT & COMPILATION)
- DIAGNOSTIC & ANALYSE INITIALE (AVANT CORRECTION)
- STRATÉGIE DE CORRECTION & BONNES PRATIQUES
- VALIDATION FINALE & RÉSULTATS

CONTEXTE & OBJECTIFS .

LE CONTEXTE :

- MAÎTRISE DE LA GESTION DYNAMIQUE DE LA MÉMOIRE EN LANGAGE C.
- DÉVELOPPEMENT D'UN PROGRAMME DE SIMULATION DE SESSIONS (MEMORY_APP.C) COMPORTANT DES ALLOCATIONS COMPLEXES.

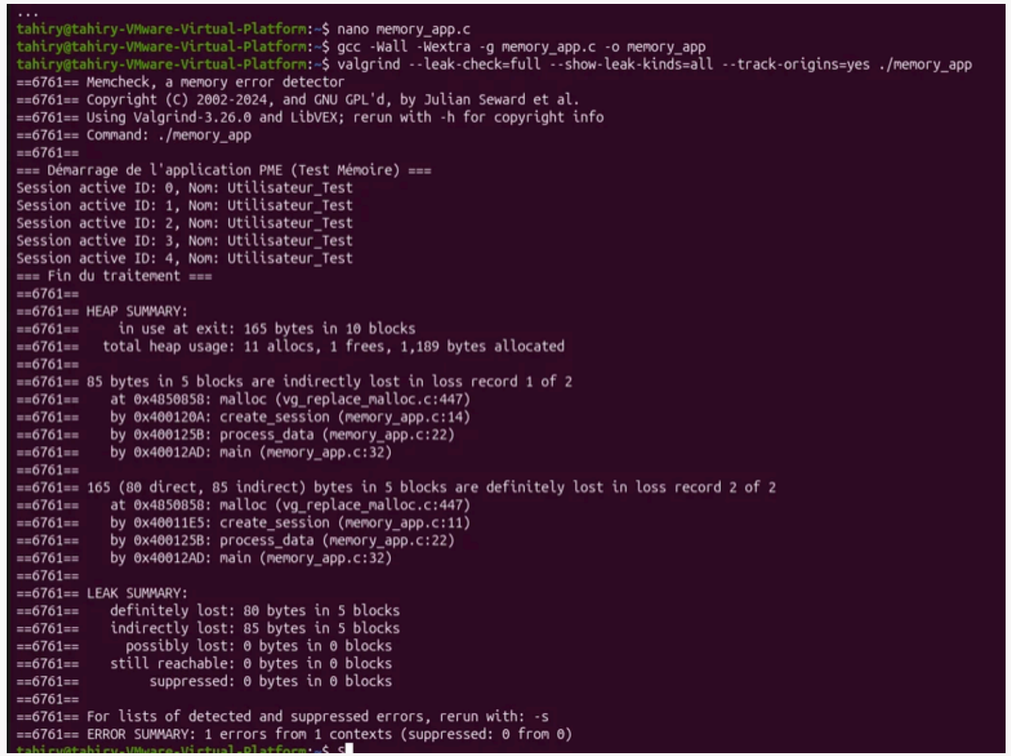
LES OBJECTIFS :

- DÉTECTER LES FUITES MÉMOIRE (MEMORY LEAKS) ET LES ACCÈS NON AUTORISÉS.
- APPLIQUER UNE MÉTHODOLOGIE DE DÉBOGAGE RIGOUREUSE.
- CORRIGER LE CODE SOURCE POUR GARANTIR UNE LIBÉRATION COMPLÈTE DES BLOCS DE MÉMOIRE (ZERO LEAK).

LA BOÎTE À OUTILS (ENVIRONNEMENT & COMPILATION)

LES OUTILS UTILISÉS :

- GCC : COMPILATION DU CODE SOURCE AVEC INTÉGRATION OBLIGATOIRE DES SYMBOLES DE DÉBOGAGE (-G) POUR LIER LES ERREURS AUX LIGNES DE CODE EXACTES.
- VALGRIND (MEMCHECK) : ANALYSEUR DYNAMIQUE POUR PISTER L'UTILISATION DU TAS (HEAP) ET REPÉRER LES BLOCS PERDUS (DEFINITELY LOST / INDIRECTLY LOST).
- GDB (GNU DEBUGGER) : INSPECTION PAS À PAS DE L'EXÉCUTION, DES REGISTRES ET DES STRUCTURES DE DONNÉES.



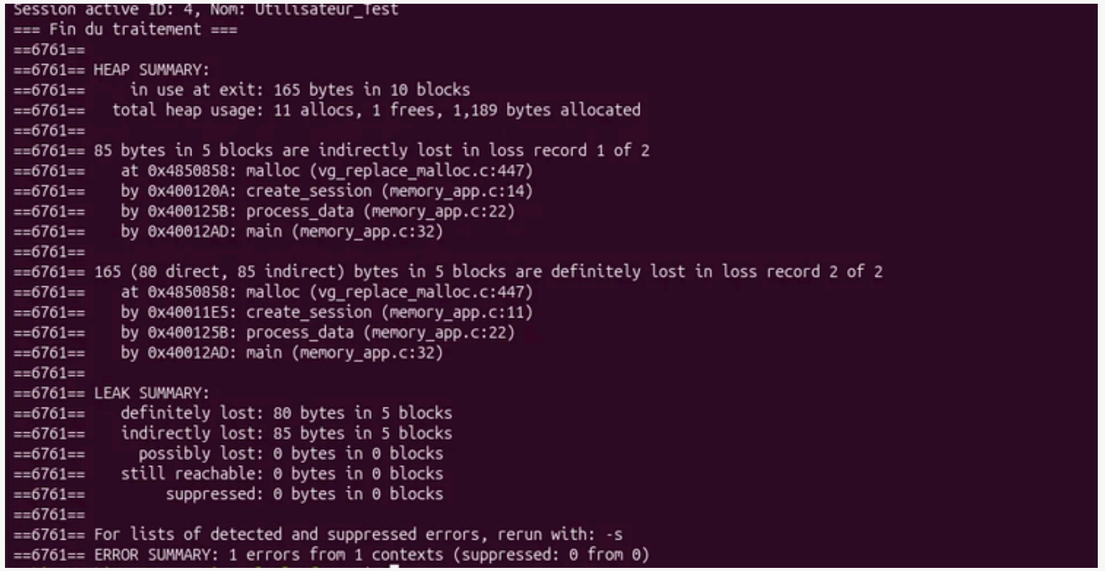
DIAGNOSTIC & ANALYSE INITIALE (AVANT CORRECTION)

LE PROBLÈME DANS LE CODE INITIAL :

- ALLOCATION DE STRUCTURES (MALLOC) ET DE CHAÎNES DE CARACTÈRES SANS INSTRUCTION DE LIBÉRATION (FREE) CORRESPONDANTE.

RÉSULTAT DU RAPPORT VALGRIND :

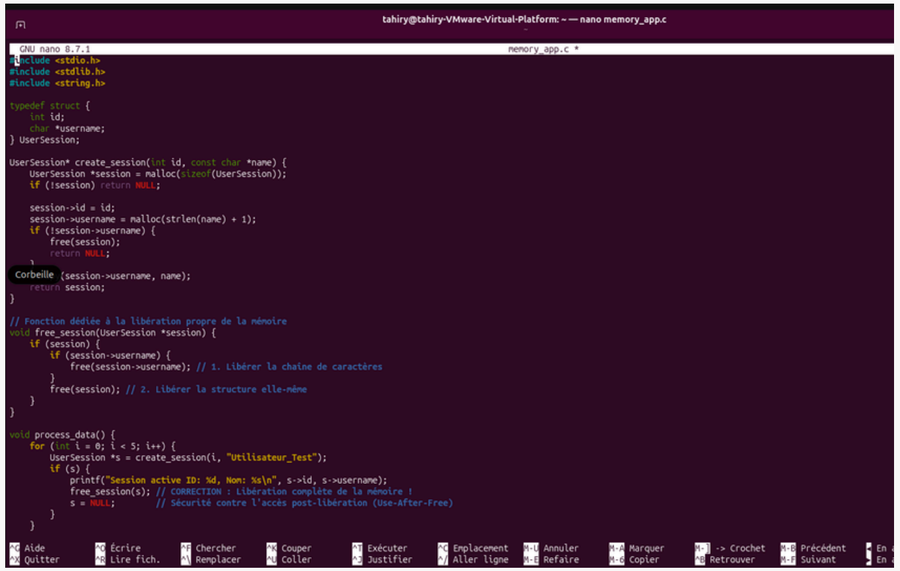
- ❌ 11 ALLOCATIONS POUR 1 SEULE LIBÉRATION.
- ❌ 80 OCTETS DÉFINITIVEMENT PERDUS ET 85 OCTETS INDIRECTEMENT PERDUS DANS 5 BLOCS NON LIBÉRÉS.



STRATÉGIE DE CORRECTION & BONNES PRATIQUES

LA SOLUTION APPORTÉE :

- CRÉATION D'UNE FONCTION DÉDIÉE AU NETTOYAGE COMPLET (FREE_SESSION()) POUR LIBÉRER D'ABORD LA MÉMOIRE INTERNE (USERNAME) PUIS LA STRUCTURE ELLE-MÊME.
- RÉINITIALISATION SÉCURISÉE DES POINTEURS À NULL APRÈS LIBÉRATION POUR CONTRER LES RISQUES DE TYPE USE-AFTER-FREE.




```c
// Fonction dédiée à la libération propre de la mémoire
void free_session(UserSession *session) {
    if (session) {
        if (session->username) {
            free(session->username); // 1. Libérer la chaîne de caractères
        }
        free(session); // 2. Libérer la structure elle-même
    }
}
```

VALIDATION FINALE & RÉSULTATS

LE VERDICT DE VALGRIND (APRÈS CORRECTION) :

```
tahiry@tahiry-VMware-Virtual-Platform:~$ gcc -Wall -Wextra -g memory_app.c -o memory_app
tahiry@tahiry-VMware-Virtual-Platform:~$ valgrind --leak-check=full ./memory_app
==6944== Memcheck, a memory error detector
==6944== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==6944== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==6944== Command: ./memory_app
==6944==
=== Démarrage de l'application PME (Test Mémoire Corrigé) ===
Session active ID: 0, Nom: Utilisateur_Test
Session active ID: 1, Nom: Utilisateur_Test
Session active ID: 2, Nom: Utilisateur_Test
Session active ID: 3, Nom: Utilisateur_Test
Session active ID: 4, Nom: Utilisateur_Test
=== Fin du traitement ===
==6944==
==6944== HEAP SUMMARY:
==6944==     in use at exit: 0 bytes in 0 blocks
==6944==   total heap usage: 11 allocs, 11 frees, 1,189 bytes allocated
==6944==
==6944== All heap blocks were freed -- no leaks are possible
==6944==
==6944== For lists of detected and suppressed errors, rerun with: -s
==6944== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
tahiry@tahiry-VMware-Virtual-Platform:~$
```